



Figure 1: Design flow

GTKWave is a fully featured GTK+ based waveform viewer, which is used to view the simulated output of the Verilog code.

Installation

These are the steps to install iverilog and GTKWave in Ubuntu.

```

sudo add-apt-repository ppa:team-electronics/ppa
sudo apt-get update
sudo apt-get install verilog gtkwave

```

An example

It is customary to start with 'Hello World'. So given below is the code for a 'Hello World' program in Verilog.

```

module helloworld;
  initial
  begin
    $display("Hello, World");
    $finish;
  end
endmodule

```

Save this file as *hello.v*. Now run the following command to compile *hello.v*:

```
iverilog -o hello hello.v
```

The results of this are placed in the file 'hello', because the '-o' flag tells the compiler where to place the compiled result. This file can be executed with the following command:

```
vvp hello
```

...and it will run the program.

Verilog is not a language used for printing 'Hello world'. So let's take a look at a logic design program. Let us design a 4-bit

full adder. A 4-bit adder will take two 4-bit values, A and B, and a carry in. The outputs will be a 4-bit sum and a 1-bit carry out.



Note: Readers are requested to read and understand digital electronics principles so as to figure out how adders and other digital circuits work.

The code for a 4-bit adder is as follows. Save it as *adder.v*

```

module half_adder(
    output S,C,
    input A,B
);
xor(S,A,B);
and(C,A,B);
endmodule

module full_adder(
    output S,Cout,
    input A,B,Cin
);
wire s1,c1,c2;
half_adder HA1(s1,c1,A,B);
half_adder HA2(S,c2,s1,Cin);
or O61(Cout,c1,c2);
endmodule

module ripple_adder_4bit(
    output [3:0] Sum,
    output Cout,
    input [3:0] A,B,
    input Cin
);
wire c1,c2,c3;
full_adder FA1(Sum[0],c1,A[0],B[0],Cin),
FA2(Sum[1],c2,A[1],B[1],c1),
FA3(Sum[2],c3,A[2],B[2],c2),
FA4(Sum[3],Cout,A[3],B[3],c3);
endmodule

module adder_tb;
// Inputs
reg [3:0] A;
reg [3:0] B;
reg Cin;
// Outputs
wire [3:0] Sum;
wire Cout;

```

We will need a test bench to simulate this code. A test bench is a program that will call an instance of our adder module, and it will supply the adder inputs to generate the output. Let us save this file as *adder_tb.v*

```

module adder_tb;
// Inputs
reg [3:0] A;
reg [3:0] B;
reg Cin;
// Outputs
wire [3:0] Sum;
wire Cout;

```